

NUMBER OF ISLANDS

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 21, 2026

© 2026 Arpit Raj. All rights reserved.

Experia

Zomato

1. Problem Overview

This problem is a classic grid traversal question. The goal is to find the total number of connected components in the grid.

- **1** → Represents **Land**
- **0** → Represents **Water**
- **Pattern**: Connected Component in Grid

2. Visual Examples

Example 1

1	1	1	1	0
1	1	0	1	0
1	1	0	0	0
0	0	0	0	0

Output = 1

Example 2

1	1	0	0	0
1	1	0	0	0
0	0	1	0	0
0	0	0	1	1

Output = 3

3. Algorithm Strategy

Traverse the entire grid. If you encounter a cell containing '1' (Land):

1. Increment the answer count: `ans++`
2. Mark all connected land cells as visited (using BFS or DFS).

Pro Tip (Marking Visited Cells):

In order to mark a cell as visited, you can either:

- Make a separate `visited` array.
- Or simply make the land into water (or change it to a character like 's' or -1), meaning it's already counted. This saves space!

| 4. Code Implementation

Approach 1: Breadth-First Search (BFS)

```

class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {

        int n=grid.size();
        int m=grid[0].size();
        vector<pair<int,int>> dir{{-1,0},{1,0},{0,1},{0,-1}};
        queue<pair<int,int>> q;

        int ans=0;
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){

                if(grid[i][j]=='1'){
                    ans++;
                    grid[i][j]='s'; //to mark it as visited
                    q.push({i,j});
                }

                while(!q.empty()){
                    int r=q.front().first;
                    int c=q.front().second;
                    q.pop();

                    for(auto it:dir){
                        int nr=r+it.first;
                        int nc=c+it.second;

                        if(nr>=0 && nc>=0 && nr<n && nc<m){
                            if(grid[nr][nc]=='1'){
                                grid[nr][nc]='s';
                                q.push({nr,nc});
                            }
                        }
                    }
                }
            }
        }

        return ans;
    }
};

```

Approach 2: Depth-First Search (DFS)

```

class Solution {
public:

    vector<pair<int,int>> dir={{-1,0},{0,1},{1,0},{0,-1}};

    void dfs(int r,int c,vector<vector<char>>& grid){

        //mark current cell as visited
        grid[r][c]='s';

        int n=grid.size();
        int m=grid[0].size();

        for(auto it:dir){

            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nr<n && nc>=0 && nc<m){

                if(grid[nr][nc]=='1'){

                    dfs(nr,nc,grid);

                }

            }

        }

    }

    int numIslands(vector<vector<char>>& grid) {

        int n=grid.size();
        int m=grid[0].size();

        int ans=0;

        for(int i=0;i<n;i++){

            for(int j=0;j<m;j++){

                if(grid[i][j]=='1'){

                    ans++;

                }

            }

        }

    }
}

```

```
        dfs(i,j,grid);  
    }  
}  
}  
return ans;  
}  
};
```